

Lecture 2: Supervised Learning, Linear Regression, maybe Regularized Regression

Max G'Sell
Machine Learning I: 46-926

November 2, 2020

Announcements

- ▶ Current office hour schedule:

- ▶ Monday, 10:20am: Me
- ▶ Tuesday, 5pm: Vaibhav
- ▶ Tuesday, 8pm: Wanshan

How does this look? Feedback welcome.

- ▶ First homework is due Wednesday *before class*.
 - ▶ Comment about gradescope

Today: Finishing basics, Linear Regression and Regularization

- ▶ Finishing up basic supervised learning, bias/variance tradeoff
- ▶ Returning to linear regression
- ▶ Extending linear regression to work in high dimensions
 - ▶ Ridge Regression
 - ▶ Lasso

Review: Supervised learning setup

We are going to focus on prediction ($Y \in \mathbb{R}$) for a little while.

We observe independent draws (x, y) from a fixed distribution $\mathcal{P}$, with $x \in \mathbb{R}^p$. We observe a training set of n examples,

$(x_1, y_1), \dots, (x_n, y_n)$. From these, we want to build a function $\hat{\mu}(x)$ that takes vectors $X \in \mathbb{R}^p$ to guesses at Y .

We measure the quality of our predictions with a *Loss function* $\mathcal{L}(Y, \hat{\mu}(X))$, which is larger for less favorable guesses.

We try to construct $\hat{\mu}$ to minimize the *expected loss*. For example:

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) = \mathbb{E}$$

Review: Regression function

We saw two important results about minimizing squared error loss last class.

The Regression Function. If we were given the generating distribution $\mathcal{P}$ that (X, Y) pairs are drawn from, then the best possible function we could choose is

$$\mu(x) =$$

The best we can do, once we have seen x , is to look at the conditional distribution of Y given that $X = x$, and take the mean.

Again, this is only true for squared-error loss! (Homework 1 Problem 2)

Review: Bias-variance decomposition

If we don't know $\mathcal{P}$, then we have to use our training set to estimate our function $\hat{\mu}$.

In what ways can we be wrong?

$$\begin{aligned}\mathbb{E} [\mathcal{L}(Y, \hat{\mu}(X)) \mid X = x] &= \mathbb{E} \left[(Y - \hat{\mu}(X))^2 \mid X = x \right] \\&= \sigma_x^2 + (\mathbb{E} [\hat{\mu}(X) \mid X = x] - \mu(x))^2 \\&\quad + \mathbb{E} \left[(\hat{\mu}(X) - \mathbb{E} [\hat{\mu}(X) \mid X = x])^2 \mid X = x \right] \\&= \sigma_x^2 + \text{bias}(\hat{\mu}(x))^2 + \text{var}(\hat{\mu}(x))\end{aligned}$$

Bias-variance decomposition

$$\mathbb{E} \left((Y - \hat{\mu}(X))^2 \mid X = x \right) = \sigma_x^2 + \text{bias}(\hat{\mu}(x))^2 + \text{var}(\hat{\mu}(x))$$

This gives you a way of thinking about where errors in predictions come from.

If your estimator is too inflexible to approximate the true mean function, it is too

If your estimator
adapts too much to individual training samples and over fits, it is too

We will see many examples.

Approximating the regression function

We know that the best we could do is use $\mu(x) = \mathbb{E}(Y|X = x)$ as our predictor, but we don't have access to it.

Instead, we have a sample $(x_1, y_1), \dots, (x_n, y_n)$ of examples. We want to use these examples to build an estimate $\hat{\mu}(x)$.

We will see many ways of constructing such an estimate. We'll begin with the simplest: Linear Regression.

Example: Linear regression

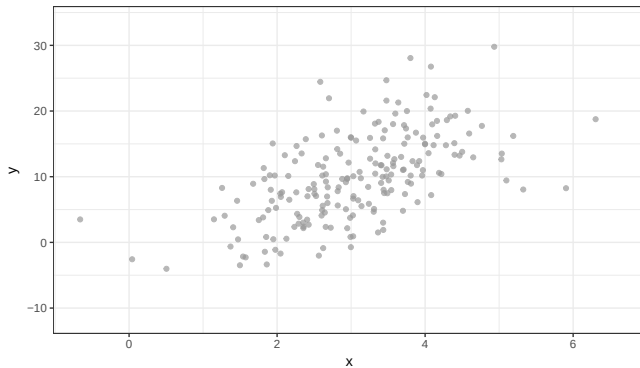
For machine learning purposes, we think of regression as a very simple way to approximate $\mathbb{E}(Y|X)$. While $\mathbb{E}(Y|X)$ could be any function, we decide to approximate it as a linear function of the predictors.

$$\begin{aligned}\hat{\mu}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

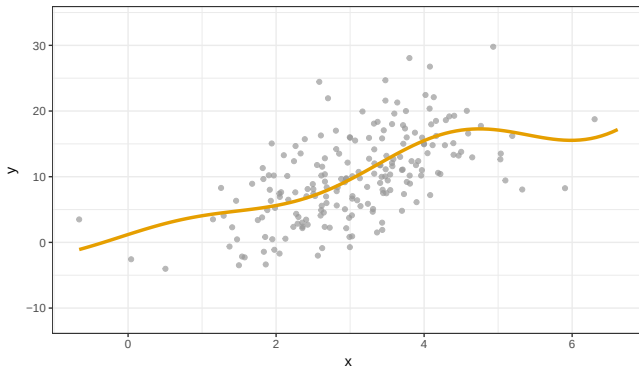
We usually estimate $\hat{\beta}$ using least squares, as you learned. Assuming a linear model simplifies things tremendously, but may also cause its own problems.

Our view of regression for prediction is quite a bit different from the common statistical view of regression. *We don't need to believe that the linear model assumptions are at all true!* We just hope it makes good guesses.

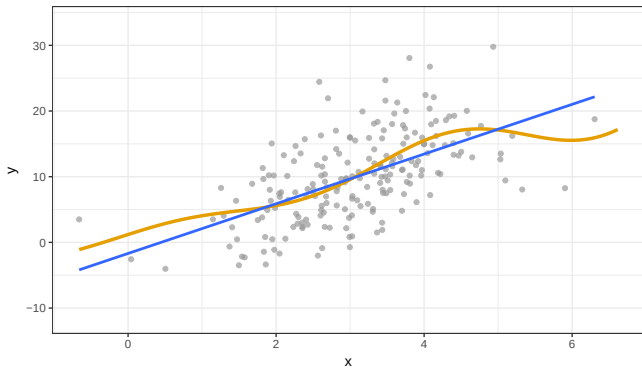
Regression function demo



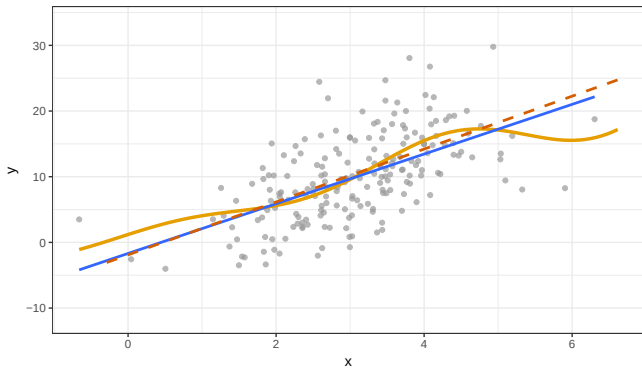
Regression function demo



Regression function demo



Regression function demo



Linear regression

$$\begin{aligned}\hat{\mu}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

If the true $\mathbb{E}(Y|X)$ is very far from linear, this model will be highly *biased*, since even $\mathbb{E}\hat{\mu}(X)$ will never be able to match $\mathbb{E}(Y|X)$.

Even with infinite data, the best linear model would still be terrible. You learned (and we will discuss) much more flexible models to avoid this problem.

We will have other problems even when the linear model is correct!

Review: Linear Regression

You've already seen a statistical perspective on linear regression last Mini in Data Science.

Suppose that we observe n copies of $(X_1, \dots, X_p, Y)$. We can fit the model

$$Y = \beta_0 + \beta_1 X_1 + \dots \beta_p X_p + \varepsilon,$$

by minimizing the RSS:

$$\begin{aligned} RSS(\beta) &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \end{aligned}$$

(Note: I'm not going to bother with the intercept after this to keep notation pleasant, you can just pretend that $X_1 = 1$.)

Review: regression with multiple predictors

We will almost always use the matrix formulation of linear regression. The same notation will be useful in other topics.

Suppose that we observe n copies of $(X_1, \dots, X_p, Y)$. We can stack all the X values into a matrix $\mathbf{X}$,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

Note that the rows $x_i \in \mathbb{R}^p$, $x_i = (x_{i1}, \dots, x_{ip})$, are the individual observations, and the columns $\mathbf{x}_j \in \mathbb{R}^n$, $\mathbf{x}_j = (x_{1j}, \dots, x_{nj})^T$, are all the observations of a particular variable.

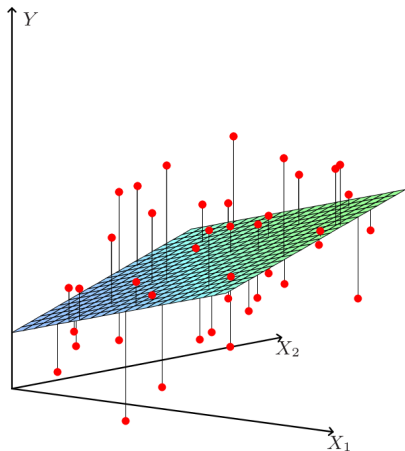
We can then write the corresponding linear model for all the Y very simply as $Y = X\beta$, or expanded,

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}$$

Why do we care? The linear algebra reformulation of the problem gives us geometric intuition into regression, which will carry over into more complicated ML procedures.

(It will also make derivations of classic statistical results much easier, though we won't get into that here.)

Two Pictures: p -space



$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Views the observations

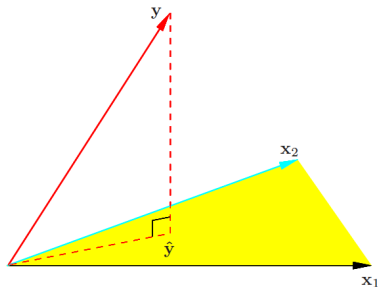
$(x_{i1}, \dots, x_{ip}, y_i)$ as points in $\mathbb{R}^p$

$$y_i = \sum_{j=1}^p x_{ij} \beta_j = x_i^T \beta$$

Easy to see:

- ▶ What a new observation does
- ▶ How well individual y_i are approximated

Two Pictures: n -space



$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Views the columns $(x_{1j}, \dots, x_{nj})$ as vectors in $\mathbb{R}^n$, along with $(y_1, \dots, y_n)$ and $(\varepsilon_1, \dots, \varepsilon_n)$

$$Y = \sum_{j=1}^n X_j \beta_j$$

Easy to see:

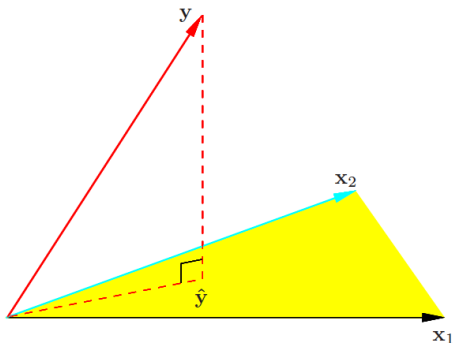
- ▶ How to fit our model
- ▶ What happens when new variables are added?
- ▶ How the error ε interacts with dimension?

Minimizing the RSS now means minimizing

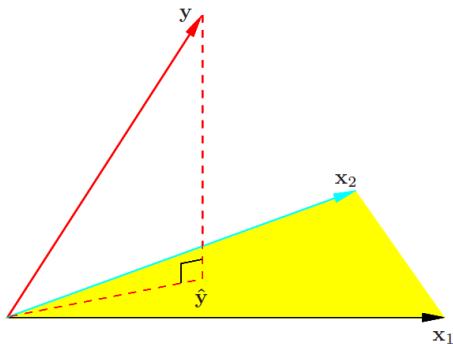
$$\begin{aligned} RSS(\beta) &= \sum_{i=1}^n \left(y_i - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \\ &= \\ &= \end{aligned}$$

Let's get an intuition for how $\hat{Y} = X\beta$ behaves.

We can write $\hat{Y} = X\hat{\beta} = \sum_{j=1}^p \hat{\beta}_j X_j$. The last term is clearly a linear combination of the columns of X , so we know that $\hat{Y}$ lies in the column span of X (a linear subspace).



(ESL pg. 46)



$\hat{Y}$ minimizes $\|Y - \hat{Y}\|_2^2$, the euclidian distance from Y to $\hat{Y}$.

Since $\hat{Y}$ is confined to the column space, this will be minimized at the closest point to Y in the subspace, meaning that $\hat{Y}$ is the orthogonal projection of Y onto the column span of X .

So $\hat{Y} = P_X Y$.

If I claim to you that

$$P_X =$$

is the orthogonal projection operator onto the column span of X , then

$$\hat{Y} =$$

and therefore the minimizing parameters are given by

$$\hat{\beta} =$$

We usually write $H = X(X^T X)^{-1} X^T$ and call this the **hat** matrix. Note that $\hat{Y} = HY$ is a linear transformation. You showed that H was symmetric and idempotent last Mini, which means that it is an orthogonal projection.

Regression: Traditional statistics vs. ML

Suppose that you observe your data X, Y , and fit a vector of coefficients $\hat{\beta}$. In traditional statistics you would do things like:

- ▶ Produce an interval $[L, U]$ that is likely to contain the next Y_i (prediction intervals)
- ▶ Determine how large β really is in the “true” model that generated the data. (Confidence intervals, hypothesis testing)

Regression: Traditional statistics vs. ML

To answer these questions, you need:

1. A model you believe for generating data, say:

$$Y_i = x_i^T \beta + \varepsilon_i$$

where ε is mean zero and variance σ^2 , and independent across observations. Or maybe $\varepsilon_i \sim N(0, \sigma^2)$

This leads us to worry about diagnostics like residual plots, which might show that our linear model is wrong!

2. Distributions of $\hat{\beta}$ for given true β . Example:

$$\hat{\beta} \sim N(\beta, \sigma^2(X^T X)^{-1})$$

A variety of results of this sort exist for different model assumptions.

Regression: Traditional statistics vs. ML

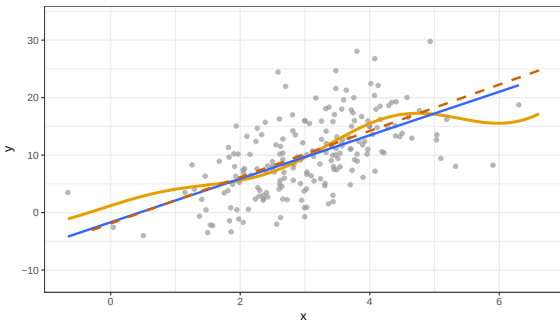
To get satisfying answers, you also worry about things like colinearity, meaning high correlations between predictors.

This is not really a problem for the math, but it tends to lead to difficulties in interpretation and in inference.

A prediction perspective

We care about all of the things above because we are trying to say something about the underlying “true” model.

From a prediction perspective, we might not really care about that true model!



Goals in modeling

In statistics and machine learning, we might fit a function $\hat{f}(x)$ to approximate a target $f(x)$. We could have several goals:

- ▶ Inference: Ask questions about $f(x)$.
- ▶ Interpretability: Ask questions about $\hat{f}(x)$
- ▶ Prediction: Guess Y using $\hat{f}(X)$.

Much of statistical inference focused on the first one. We will primarily care about the last two.

Goals in modeling

You have heard many horror stories about *colinearity* in regression. Suppose you fit a regression of Y on $X_1, X_2, \dots$, but X_1 and X_2 are almost the same.

What happens to inference for linear regression?

What happens to predictions based on linear regression?

Linear regression as a predictor

Ok, so suppose that we use linear regression just to make predictions. What will happen?

You can obtain estimates $\hat{Y} = X\hat{\beta}$, and they might work well sometimes, even when the linear model is not true!

In particular, we know that least squares has nice properties when the linear model is true – e.g., it is unbiased and has minimum variance among unbiased, linear estimates (Gauss-Markov Theorem).

However...

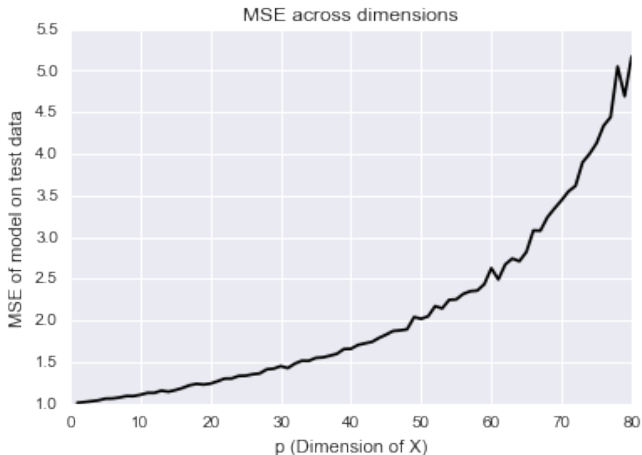
Linear regression in high dimensions

What happens when we have a bunch of variables? (Big data!)

We know that a linear model may perform terribly if the true function is far from linear.

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

Linear regression in high dimensions



This is a simulation from a true linear model with independent Gaussian errors – the dream!

Linear regression in high dimensions

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

As dimension increases, the prediction error increases (super-)linearly! When dimension is bigger than the number of observations,

What is going on, and how can we fix it?

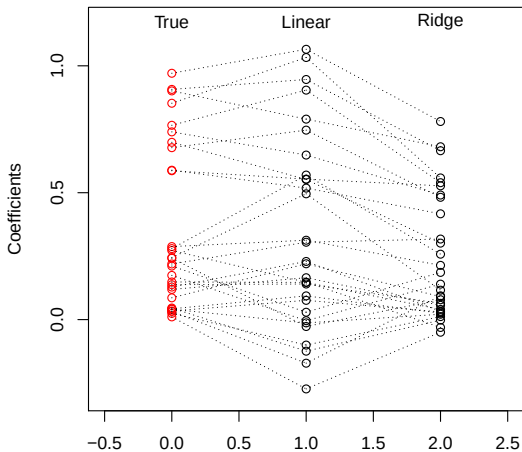
Fixing linear regression in high dimensions

How do we reduce this error?

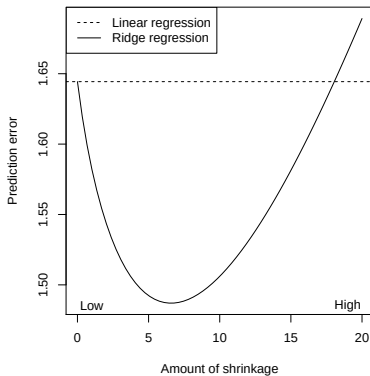
$$\mathbb{E} \left((Y - \hat{\mu}(X))^2 \right) = \sigma_x^2 + \text{Bias}(\hat{\mu}(X))^2 + \text{Var}(\hat{\mu}(X))$$

Example: visual representation of ridge coefficients

A visual representation of the **ridge regression** coefficients for the same example ($n = 50$, $p = 30$, and $\sigma^2 = 1$; 10 large true coefficients, 20 small) at $\lambda = 25$:



Does it work?



Linear regression:

Squared bias = 0

Variance ≈ 0.633

Pred. error $\approx 1 + 0.633$
 ≈ 1.633

Ridge regression, at its best:

Squared bias ≈ 0.077

Variance ≈ 0.403

Pred. error $\approx 1 + 0.077 + 0.403$
 ≈ 1.48

This is upsetting! And amazing!

Ridge regression

Ridge regression is like least squares but shrinks the estimated coefficients towards zero. Given a response vector $y \in \mathbb{R}^n$ and a predictor matrix $X \in \mathbb{R}^{n \times p}$, the ridge regression coefficients are defined as

$$\begin{aligned}\hat{\beta}^{\text{ridge}} &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \\ &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}\end{aligned}$$

Ridge regression

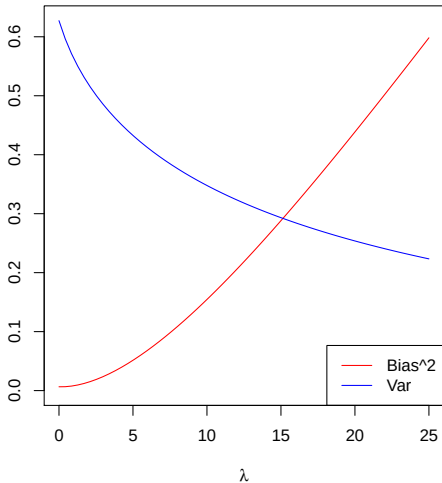
$$\begin{aligned}\hat{\beta}^{\text{ridge}} &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \\ &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}\end{aligned}$$

Here $\lambda \geq 0$ is a **tuning parameter**, which controls the strength of the penalty term. Note that:

- ▶ When $\lambda = 0$,
- ▶ When $\lambda = \infty$,
- ▶ For λ in between, we are balancing two ideas: fitting a linear model of y on X , and shrinking the coefficients

Example: bias and variance of ridge regression

Bias and variance for our last example ($n = 50$, $p = 30$, $\sigma^2 = 1$; 10 large true coefficients, 20 small):



Important details: centering

When including an **intercept** term in the regression, we usually leave this coefficient **unpenalized**.

Otherwise we could add some constant amount c to the vector y , and this would not result in the same solution.

Hence ridge regression with intercept solves

$$\hat{\beta}_0, \hat{\beta}^{\text{ridge}} = \underset{\beta_0 \in \mathbb{R}, \beta \in \mathbb{R}^p}{\operatorname{argmin}} \|y - \beta_0 \mathbf{1} - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

If we center the columns of X , then the intercept estimate ends up just being $\hat{\beta}_0 = \bar{y}$, so we usually just assume that y, X have been centered and don't include an intercept

Important details: scaling

Also, the penalty term $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$ is unfair if the predictor variables are **not on the same scale**. (Why?)

Therefore, if we know that the variables are not measured in the same units, we typically **scale** the columns of X (to have sample variance 1), and then we perform ridge regression

We see that squishing the coefficients toward zero gives:

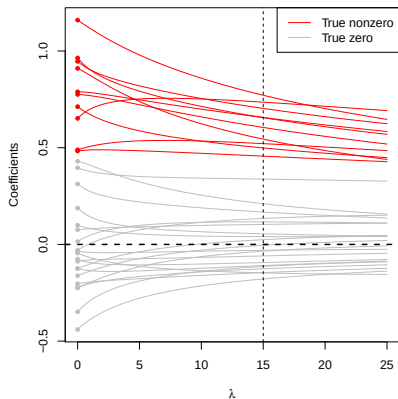
- ▶ A decrease in variance!
- ▶ A decrease in MSE!
- ▶ An increase in bias...

However, ridge regression:

- ▶ Seems quite harsh on large, important coefficients.
- ▶ Leaves us with a large linear model, which is not that interpretable when we have many variables.

Ridge regression coefficients

Ridge regression coefficients, plotted against λ :



The red paths correspond to the true nonzero coefficients; the gray paths correspond to true zeros. The vertical dashed line at $\lambda = 15$ marks the point above which ridge regression's MSE starts losing to that of linear regression

An important thing to notice is that the gray coefficient paths are not **exactly zero**; they are shrunk, but still nonzero

The lasso

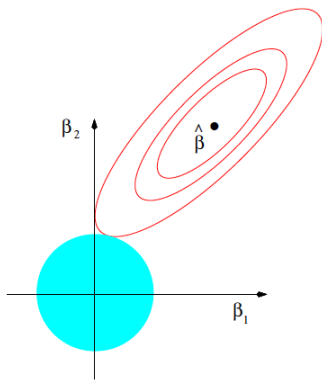
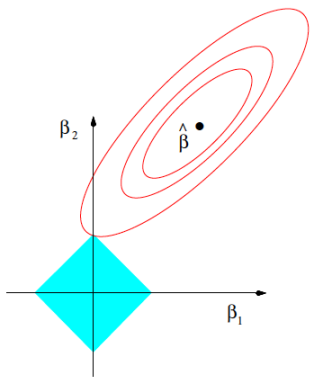
The **lasso**¹ estimate is defined as

$$\begin{aligned}\hat{\beta}^{\text{lasso}} &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^p |\beta_j| \\ &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_1}_{\text{Penalty}}\end{aligned}$$

The squared ℓ_2 penalty $\|\beta\|_2^2$ of ridge regression, has been replaced by an ℓ_1 penalty $\|\beta\|_1$. Even though these problems look similar, their solutions behave very differently

Note the name “lasso” is actually an acronym for: Least Absolute Selection and Shrinkage Operator

¹Tibshirani (1996), “Regression Shrinkage and Selection via the Lasso”



$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

The **tuning parameter** λ controls the strength of the penalty.

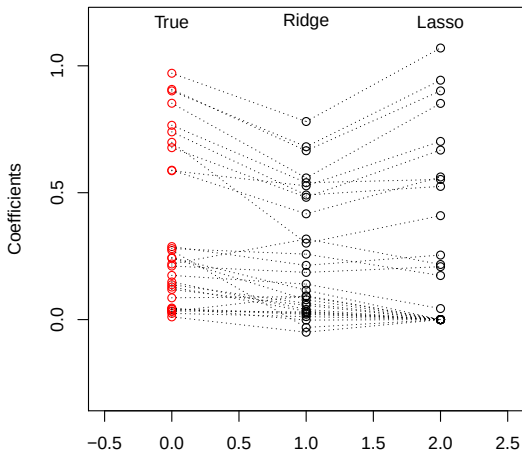
- ▶ If $\lambda = 0$:
- ▶ As $\lambda \rightarrow \infty$:

For λ in between these two extremes, we are balancing two ideas: fitting a linear model of y on X , and shrinking the coefficients. But the nature of the ℓ_1 penalty causes some coefficients to be shrunk to **zero exactly**

This leads to **variable selection**!

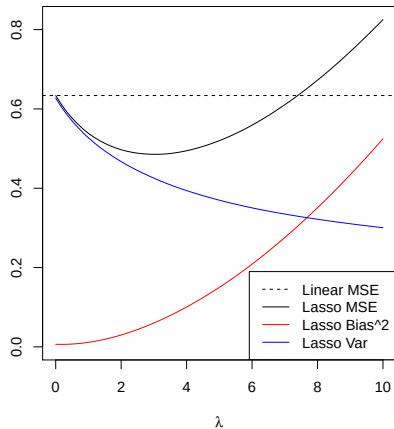
Example: visual representation of lasso coefficients

Our running example from last time: $n = 50$, $p = 30$, $\sigma^2 = 1$, 10 large true coefficients, 20 small. Here is a visual representation of lasso vs. ridge coefficients (with the same degrees of freedom):



Example: subset of small coefficients

Example: $n = 50$, $p = 30$; true coefficients: 10 large, 20 small



Advantages of sparsity

- ▶ Interpretability: We can understand what the model relies on for prediction (understanding $\hat{f}$)
- ▶ We might gain some insight into the underlying data (though not causally) (helping to understand f)
- ▶ If we're building a predictive score, we can measure fewer things in the future (simpler $\hat{f}$ to apply later)

Important details

Just like in ridge regression:

- ▶ We don't penalize our intercept, if one is included. Again, columns are often centered and then intercept omitted.
- ▶ The penalty term $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$ is not fair if the predictor variables are **not on the same scale**. Hence, we often scale the columns of X to have variance 1.

How is it actually solved?

[REMINDER: SHOW SOFTWARE DEMOS]

Interview question

Suppose that you regress Y on X_1, X_2 using both ridge regression and the lasso (with $\lambda > 0$). If $X_1 = X_2$, what does:

The solution to ridge look like

The solution to lasso look like

Simple case: Orthogonal X

Consider the very simple case where $X = I$. We can examine how both ridge regression and the lasso behave on this data. This is easy, because the variables do not interact with one another, but it gives an intuition for their more general behavior.

When $X = I$, note that the RSS becomes

$$\|y - X\beta\|_2^2 = \|y - \beta\|_2^2 = \sum_{i=1}^n (y_i - \beta_i)^2.$$

Simple case: Orthogonal X

The corresponding penalized forms then become

Ridge:

$$\hat{\beta}_{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_i)^2 + \lambda \beta_i^2$$

Lasso:

$$\hat{\beta}_{\text{lasso}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_i)^2 + \lambda |\beta_i|$$

We can minimize separately for each i !

For ridge regression, differentiating yields

and therefore

$$\hat{\beta}_i =$$

Thus the estimates from ridge regression correspond to the least squares estimates reduced by a constant multiple.

Note that this can never give zero coefficients, and that it penalizes large coefficients quite a bit.

For the lasso, it turns out that the minimizer of

$$(y_i - \beta_i)^2 + \lambda|\beta_i|$$

is given by

$$\hat{\beta}_i =$$

Thus estimates from ridge regression correspond to shrinking the least squares estimates toward zero by an additive constant (without crossing zero).

This can give zero coefficients, and is also relatively harsher on small coefficients than larger ones. It appears to be better suited to sparse settings.

Regularized regression

Least squares solves the problem:

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2$$

In regularized regression, we add a penalty to this squared error loss term

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2 + P_{\lambda}(\beta)$$

You've seen several examples: $P_{\lambda}(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_{\lambda}(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both “pull” β toward 0, causing it to be less variable. However, by making β smaller than the least squares estimate, they add bias.

The λ parameter determines how strong this penalty is. A larger λ leads to a higher bias and a smaller variance.

Regularized regression

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2 + P_{\lambda}(\beta)$$

$P_{\lambda}(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_{\lambda}(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both “pull” β toward 0, causing it to be less

However, by making β smaller than the least squares estimate, they add

The λ parameter determines how strong this penalty is. A larger λ leads to a higher and a smaller

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

We will see at least:

- ▶ Logistic regression with a lasso penalty
- ▶ The elastic net

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

Other classic examples:

- ▶ The group lasso: setting groups of variables to zero
- ▶ Graphical lasso: estimates networks of dependence between variables
- ▶ Fused lasso: estimates piecewise-constant functions
- ▶ Trend filtering: estimates piecewise-polynomial functions
- ▶ Low-rank matrix decomposition: estimate a low-rank approximation to a matrix
- ▶ Sparse additive models
- ▶ Vector auto-regressive models